1. **Are you able to process the entire Qualitas Corpus? If not, what are the main issues (think in terms of data processing and storage) that causes the cljDetector to fail? How may you modify the application to avoid these issues?**

The current system's design causes high resource consumption:

- High RAM Usage: The implementation reads every file, splits it into chunks, and holds all pre-processed data in RAM before writing to the database. This approach leads to a substantial memory peak which risks exhausting the Java Virtual Machine's heap space, particularly with large projects. The all documents are not able to be processed all together at once in fact the first time that I processed the Quality corpus it was able to store just 70k of file of the 120 detected once, the second time that I rerun the process the function process the remaining 50k files, this could be maybe due to a limitated heap space for get all the files in memory. I was able to write the remaining files just because there is a filter that check the files to be stored based on the already name of files that are stored in the database, but I recovered an issue of the detected new files the system wrote just the chunks instead of files and chunks.
- Prolonged I/O Times: The time required for persistence is excessive. Observations showed that the initial file processing (loading to RAM) took about one hour, followed by the store of the files of another hour and in the last followed by the bulk storage of all chunks, which lasted over two hours.

Data Mounting Constraint: The large size of the Qualitas Corpus necessitates mounting it as an external bind volume instead of a native Docker volume. This is due to storage limits in Docker, but it may introduce I/O overheads.

Proposed Mitigations To improve system stability and performance, the processing must be restructured to a progressive (batch) model:

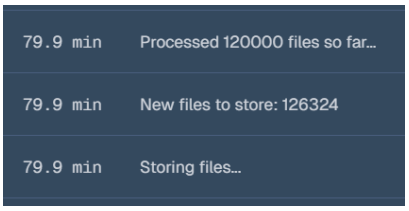
- **Sequential Batch Processing:** Instead of processing all files simultaneously, the sequence of reading, chunking, and storing should be repeated for small batches of N files. This will significantly reduce the peak RAM usage by ensuring only a fraction of the data resides in memory at any given time.
- **Progressive Writing:** Storing file and chunk data in smaller, more frequent batches will further alleviate RAM congestion and reduce the risk of I/O timeouts during massive

Positive Out Comes

Despite the acquisition challenges, the clone detection phase is now significantly faster. This improvement confirms the successful optimization of the core algorithm, particularly using the stream-of-code approach and efficient hashing.

Final Resuts after the storing the entire qualitas corpus :

After preprocessing files:



Data expected :

- 126324 files <- right count of the files
- 14.018.627 <- right count of the chunks

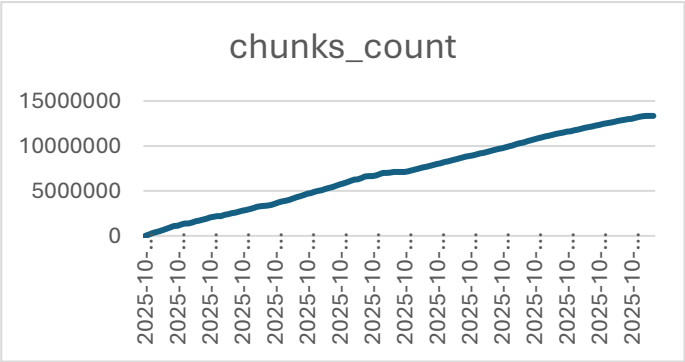
1. Report of the analysis

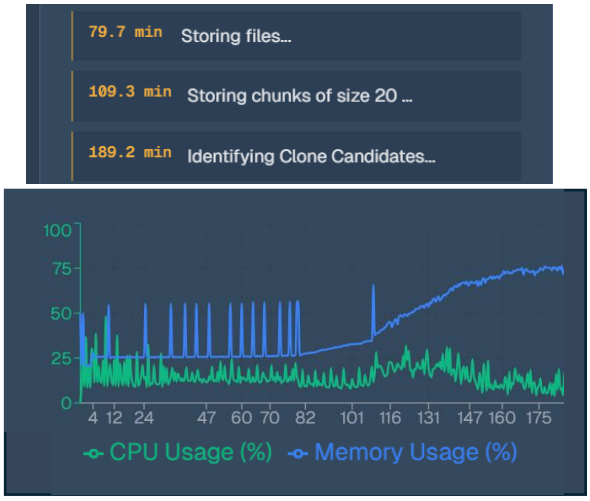
2.1 Is the time to generate chunks constant or does it vary with the number of already generated chunks?

The time to generate chunks is not constant, it varies as the process evolves.
Chunk generation occurs in a lazy manner after the files have been stored, this for not load all the chunks at once. For each processed file, chunks are generated on demand and then stored in the database. Each chunk is generated from the already processed files and contains only references to the corresponding lines and a hash code that identifies its content.

During this phase, we can observe an increase in memory usage because the chunks are created lazily, and the garbage collector is not immediately triggered to clear them.

Although memory consumption grows, the overall performance tends to improve over time. This happens because the CPU gradually optimizes how to handle chunk generation and storage through a pipeline-based execution. Once the pipeline becomes full, the phases of chunk creation and storage are better coordinated and executed in parallel.
As a result, the time required to store new chunks decreases over time, since the system effectively “learns” to work more efficiently as the pipeline stabilizes.

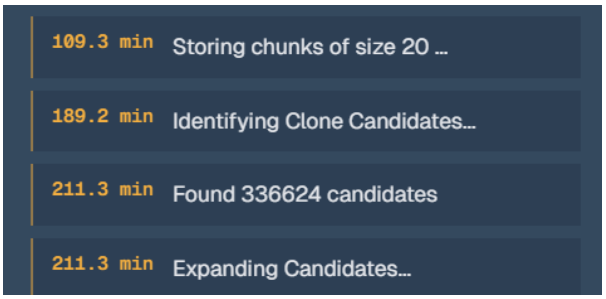


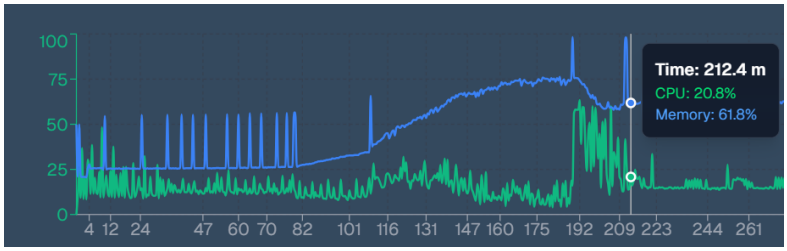


2.2 Is the time to generate clone candidates constant or does it vary with the number of already generated candidates?

The time for generating new candidates is not possible to follow in real time because is an atomic operation of MongoDB that stores all the detected candidates together. We can try to split the chunk for see the time for search of new candidates, but this means that the chunks are not compared all together for generate the candidates and load in memory all the chunks at once is not possible for big project. The generation of new candidates is based on the comparing of the hash not by the content of the chunks and at the end the candidates are the ones in the pipes that have a replica counter more than 1. In the theory we can expect that the time for generating new candidates is constant because the process doesn't have a check on all the already generated candidates. We can conclude that the time for generate new candidates is just influenced by the number of chunks to compare.

The time for get new candidates it was close to half an hour and it generate this amount of candidates, the next time that I re-tried this process and that were stored the remaining 50.000 files the values the chunks where grow from 13 million to 19 million and as unexpected result the candidates bring to over 5 million instead of the 300k generated before.

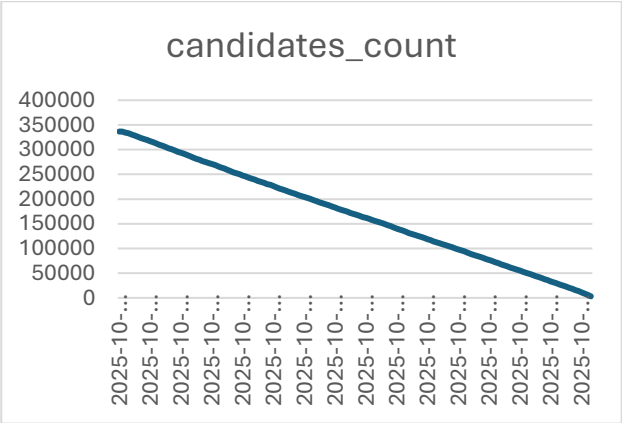


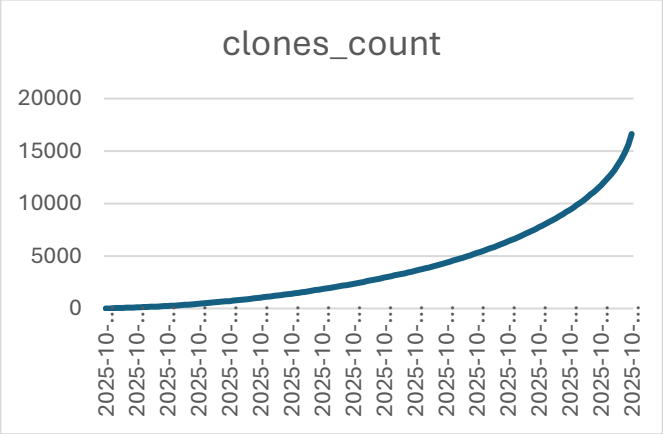


2.3 Is the time to expand clone candidates constant or does it vary with the number of already expanded clones or the number of remaining candidates?

The time for expand the candidates is close to be constant. Is processed one candidate at time and is verified if could be expanded to the other candidates and if there is one that is part of the same file and the line overlap the candidates are merged and only the biggers are saved. The process is recursive and also after the first time that a bigger candidate is merged the function recall itself for see if there are still other candidates overlapping with the actual one. During the first iteration smaller candidates are removed and it become easier to remove candidates from the detected clones, this is why we have an exponential grow through the time.

And also we can say that this process is not an heavy process for the memory, most of the work is done by mongo that fetch the overlapping clones in a fast way.





2.4 What is the average clone size? How can this be used to predict progress during the expansion phase?

```
db.clones.aggregate([
 {$unwind: "$instances" },        // expand all the instances
 { $project: {
    cloneSize: { $subtract: ["$instances.endLine", "$instances.startLine"]}
  } },
 { $group: {
    _id: null, avgCloneSize: { $avg: "$cloneSize" }   // global average
  }}])
```

```
{
  _id: null,
  avgCloneSize:
39.20629617845653
}
```

This average could be used for the model for try to expand a candidate until the size is not reached and when it does stop to find for more overlap, in this way we have a stop criteria for try to expand and find other overlapping candidates.

What is the average number of chunks per file? How can this be used to predict progress during the read/chunkify/identify candidates stages?

```
db.chunks.aggregate([
 {$group: {
    _id: "$fileName", chunkCount: { $sum: 1 }   // enumerate the chunks of a file
  } },
 { $group: {
    _id: null,   avgChunksPerFile: { $avg: "$chunkCount" }  // get the average count for each file
  }}])
```

```
{
_id: null,
  avgChunksPerFile:
169.16945636120684
}
```

Does it mean that each file has 169+size_chunks lines. This information could be used for create a data space for store the file before even read the file and then just changing the hash code for each chunk. During the candidate expand instead we have a fixed number to chunk to check for the overlap and when that value is reached the search for overlap chunks could be stopped. The information could be used also for optimize system threads, queues, or worker pools by using average chunk loads as baseline assumptions for load-balancing i can have a constant size for the stream of data sending single files.

Edit Made on the project

Edit on the src.core.clj:

- Filter for not store new files on the maybe read
- Message log during the reading of files
- Ts-println for store the statistics and the message of update

Edit on the src.storage.storage.clj :

- Add Update for store the messages log on the database
- Store statistics for store the inner statistics of the system
- Get existing filenames for get the name of the stored files in the database
- File-exist for verify if the file is in the collection of the files from the db or not

Some data report :

Example on a small set of data

cloneDetector.statisticsHistory-practical-example.xlsx

Example on the qualitas corpus

The example show the error on the second run of the corpus showing of the 50 new files not stored, the new chunks stored and the large number of candidates detected, that could means that the new chunks stored are just duplicated chunks of the already stored chunks.

cloneDetector.statisticsHistory-QualityCorpus.xlsx